# Curso Ciencia de Datos

# Proyecto Módulo 9

# Retail Analytics Pipeline.

## FUNDAMENTOS DE BIG DATA

**Alumno Reinaldo Elgueta Ábalos**
**Abril 2026**

## Situación:

El Departamento de Analítica y Machine Learning de la empresa RetailMax, la cual maneja millones de datos diarios de ventas, solicita desarrollar un **pipeline de Big Data y Machine Learning escalable**, capaz de:

1. Procesar datos estructurados y no estructurados en entornos distribuidos.
2. Ejecutar transformaciones masivas para generar métricas de negocio.
3. Entrenar y evaluar modelos de Machine Learning sobre grandes volúmenes de datos utilizando Spark MLlib.

## Lección 1: Fundamentos de Big Data

📌 Objetivo: Reconocer el ecosistema de Big Data y sus componentes básicos.
Es decir, se busca identificar los conceptos teóricos asociados al negocio RetailMax para justificar el uso de tecnologías Apache Spark.

📍 Tareas a desarrollar:

● ***Describir las 5V de Big Data y su impacto en proyectos reales.***

Las 5V se refieren a : Volumen - velocidad - variedad - veracidad - valor.

Las 5V responderán a la descripción de cómo impactarán las 5 dimensiones en el proyecto:

1. Volumen:
   RetailMax maneja millones de transacciones diarias y grandes volúmenes de datos de navegación y reseñas. Esto excede la capacidad de almacenamiento y procesamiento de sistemas tradicionales.

2. Velocidad
   Los datos se generan en tiempo real (compras, clics). Se requiere un pipeline que pueda procesar esta información rápidamente para generar métricas de negocio oportunas.

3. Variedad
   Manejamos datos estructurados (transacciones de ventas), semi-estructurados (reseñas y calificaciones en formato JSON o CSV) y no estructurados (comportamiento de navegación).

4. Veracidad
   Es crucial limpiar los datos para asegurar que las reseñas y métricas de comportamiento sean auténticas antes de entrenar los modelos de Machine Learning.

5. Valor
   El objetivo final es clasificar usuarios para generar estrategias de marketing accionables que incrementen la rentabilidad.

● ***Identificar las fuentes de datos de RetailMax (transacciones, reseñas y navegación).***

De acuerdo con la consigna, nuestras fuentes principales serán:

1. Transacciones: Datos de ventas (Estructurados).
2. Reseñas y Calificaciones: Opiniones de clientes (Semi-estructurados).
3. Navegación: Logs de comportamiento en la plataforma e-commerce (No estructurados).

● ***Proponer un diagrama inicial de arquitectura para el proyecto.***

Para este pipeline escalable, la arquitectura recomendada sigue un flujo de Data Lake:

1. Capa de Ingesta: Captura de datos de las fuentes mencionadas.
2. Capa de Almacenamiento: Uso de formatos optimizados como Parquet.
3. Capa de Procesamiento: Apache Spark como motor principal para transformaciones masivas (RDDs y DataFrames).
4. Capa de Analítica y ML: Implementación de modelos con Spark MLlib para segmentación y clasificación

● ***Entregar un informe conceptual que sirva como base para las siguientes etapas.***

## **Lección 2:** Apache Spark - Introducción y Configuración

Respaldo de imágenes obtenidas:

```
Ecosistema Big Data: Datos cargados en RDD exitosamente.
1. Acción COUNT: El dataset contiene 10000 transacciones.

2. Acción TAKE (Primeros 5 registros):
(0, 'user_58', 429.28, 'Juguetes')
(1, 'user_929', 217.71, 'Electrónica')
(2, 'user_783', 295.41, 'Juguetes')
(3, 'user_703', 101.55, 'Electrónica')
(4, 'user_786', 439.36, 'Ropa')

3. Acción FIRST: El primer registro es: (0, 'user_58', 429.28, 'Juguetes')
```

```
[OK] Conectividad validada: El RDD no está vacío y responde a consultas.
[INFO] Particiones activas: 2
```

## Lección 3: Elementos básicos de Spark (RDD, Transformaciones y Acciones)

Explicación de resultados obtenidos:

Esta lección marca la transición del almacenamiento a la computación distribuida, donde manipulamos los datos masivos de **RetailMax** para extraer valor de negocio.

Imagen obtenida luego de ejecutar la línea de códigos asociada a las indicaciones:

```
Muestra Ordenada (collect): [('JUGUETES', 499.89), ('HOGAR', 499.88), ('LIBROS', 499.86), ('LIBROS', 499.83), ('HOGAR', 499.82)]



Suma Total (sum): 2430104.07
Promedio (mean): 298.98
Desviación Estándar (stdev): 115.29



--- Linaje del RDD (DAG) ---
(2) PythonRDD[53] at RDD at PythonRDD.scala:56 []
 |  MapPartitionsRDD[48] at mapPartitions at PythonRDD.scala:168 []
 |  ShuffledRDD[47] at partitionBy at NativeMethodAccessorImpl.java:0 []
 +-(2) PairwiseRDD[46] at sortBy at /tmp/ipykernel_5875/3951618426.py:30 []
    |  PythonRDD[45] at sortBy at /tmp/ipykernel_5875/3951618426.py:30 []
    |  ParallelCollectionRDD[38] at readRDDFromFile at PythonRDD.scala:297 []
```

### 1. Transformaciones Aplicadas (Procesamiento Lógico)

```
Muestra Ordenada (collect): [('JUGUETES', 499.89), ('HOGAR', 499.88), ('LIBROS', 499.86), ('LIBROS', 499.83), ('HOGAR', 499.82)]
```

De acuerdo a la primera linea del código, realizamos una cadena de transformaciones sobre los RDDs para estructurar la información:

- **Creación de Pair RDDs:** Transformamos los datos planos en estructuras de **Clave-Valor** (Categoría, Monto). Esto es esencial en Big Data para permitir agrupamientos masivos por etiquetas como 'JUGUETES' o 'HOGAR'.
- **Limpieza y Mapeo (map, filter):** Se estandarizaron las categorías a mayúsculas y se aplicaron filtros para conservar únicamente las transacciones que cumplen con criterios de relevancia analítica.
- **Ordenamiento (sortBy):** Organizamos los datos de forma descendente por monto para identificar el "Top de ventas". En tu captura, se observa que los montos más altos rondan los **499.89**.

### 2. Análisis de Resultados (Acciones Ejecutadas)

Este corresponde al segundo párrafo obtenido.

```
Muestra Ordenada (collect): [('JUGUETES', 499.89), ('HOGAR', 499.88), ('LIBROS', 499.86), ('LIBROS', 499.83), ('HOGAR', 499.82)]


Suma Total (sum): 2430104.07
Promedio (mean): 298.98
Desviación Estándar (stdev): 115.29


--- Linaje del RDD (DAG) ---
(2) PythonRDD[53] at RDD at PythonRDD.scala:56 []
 |  MapPartitionsRDD[48] at mapPartitions at PythonRDD.scala:168 []
 |  ShuffledRDD[47] at partitionBy at NativeMethodAccessorImpl.java:0 []
 +-(2) PairwiseRDD[46] at sortBy at /tmp/ipykernel_5875/3951618426.py:30 []
    |  PythonRDD[45] at sortBy at /tmp/ipykernel_5875/3951618426.py:30 []
    |  ParallelCollectionRDD[38] at readRDDFromFile at PythonRDD.scala:297 []
```

Las acciones son las que obligan a Spark a ejecutar el cómputo y devolver resultados al Driver:

- **collect (Muestra Ordenada):** Validamos que la lógica de ordenamiento fue correcta. Tu salida muestra que el top de ventas está liderado por categorías como **JUGUETES, HOGAR y LIBROS**.
- **Métricas Descriptivas:**
  - **Suma Total (sum):** $2,430,104.07. Es el volumen total de dinero procesado en esta ventana de tiempo.
  - **Promedio (mean):** $298.98. Es el gasto medio por transacción.
  - **Desviación Estándar (stdev):** $115.29. Indica una dispersión moderada, sugiriendo que las ventas son relativamente consistentes pero con suficiente variedad para segmentar usuarios en la Lección 5.

### 3. Explicación Detallada del DAG (Linaje de Datos)

El **DAG (Directed Acyclic Graph)** es el plan de ejecución que Spark construye para ser eficiente y tolerante a fallos.

Este corresponde al tercer párrafo obtenido

```
Muestra Ordenada (collect): [('JUGUETES', 499.89), ('HOGAR', 499.88), ('LIBROS', 499.86), ('LIBROS', 499.83), ('HOGAR', 499.82)]



Suma Total (sum): 2430104.07
Promedio (mean): 298.98
Desviación Estándar (stdev): 115.29


--- Linaje del RDD (DAG) ---
(2) PythonRDD[53] at RDD at PythonRDD.scala:56 []
 |  MapPartitionsRDD[48] at mapPartitions at PythonRDD.scala:168 []
 |  ShuffledRDD[47] at partitionBy at NativeMethodAccessorImpl.java:0 []
 +-(2) PairwiseRDD[46] at sortBy at /tmp/ipykernel_5875/3951618426.py:30 []
    |  PythonRDD[45] at sortBy at /tmp/ipykernel_5875/3951618426.py:30 []
    |  ParallelCollectionRDD[38] at readRDDFromFile at PythonRDD.scala:297 []
```

**Interpretación de la captura de abajo hacia arriba:**

1. **ParallelCollectionRDD[38]:** Es el punto de entrada donde los datos se dividen en las **2 particiones** que muestra tu sistema.
2. **PythonRDD[45] / PairwiseRDD[46]:** Aquí ocurren las transformaciones "estrechas" (mapas). Cada partición trabaja de forma independiente, lo que hace que Spark sea extremadamente rápido.
3. **ShuffledRDD[47]:** Es el paso más crítico. Para poder ordenar los datos por monto (sortBy), Spark debe redistribuir los datos entre las particiones. A esto se le llama **Shuffle**. Es una operación costosa pero necesaria para obtener un orden global.
4. **PythonRDD[53]:** Es el estado final del RDD listo para ser consumido por las acciones estadísticas.

# Lección 4:

Respaldo de imágenes obtenidas

```
root
 |-- id_transaccion: integer (nullable = true)
 |-- id_usuario: string (nullable = true)
 |-- monto: float (nullable = true)
 |-- categoria: string (nullable = true)

+--------------+----------+------+-----------+
|id_transaccion|id_usuario| monto|  categoria|
+--------------+----------+------+-----------+
|             0|   user_58|429.28|   Juguetes|
|             1|  user_929|217.71|Electrónica|
|             2|  user_783|295.41|   Juguetes|
|             3|  user_703|101.55|Electrónica|
|             4|  user_786|439.36|       Ropa|
+--------------+----------+------+-----------+
only showing top 5 rows
```

```
+-----------+------------------+----------------------+
|  categoria|      total_ventas|cantidad_transacciones|
+-----------+------------------+----------------------+
|      Hogar|522706.61980724335|                  2000|
|   Juguetes|515979.06014442444|                  2023|
|Electrónica| 513918.2698316574|                  2006|
|     Libros| 510469.2196073532|                  1986|
|       Ropa|487864.68007564545|                  1985|
+-----------+------------------+----------------------+
```

```
Archivo Parquet generado y guardado exitosamente.
  adding: retail_data_processed.parquet/ (stored 0%)
  adding: retail_data_processed.parquet/._SUCCESS.crc (stored 0%)
  adding: retail_data_processed.parquet/.part-00000-f3f97b31-6bc4-445c-a733-d54f067d10e9-c000.snappy.parquet.crc (stored 0%)
  adding: retail_data_processed.parquet/_SUCCESS (stored 0%)
  adding: retail_data_processed.parquet/.part-00001-f3f97b31-6bc4-445c-a733-d54f067d10e9-c000.snappy.parquet.crc (stored 0%)
  adding: retail_data_processed.parquet/part-00000-f3f97b31-6bc4-445c-a733-d54f067d10e9-c000.snappy.parquet (deflated 34%)
  adding: retail_data_processed.parquet/part-00001-f3f97b31-6bc4-445c-a733-d54f067d10e9-c000.snappy.parquet (deflated 34%)
```

## Lección 5: Introducción a Machine Learning Escalable (Spark MLlib)

**"Informe Final en PDF"**

Captura de imagen del código final

```
Modelo de Regresión Logística entrenado.

Segmentación de Clientes (Centros de Cluster):
Cluster 0: Gasto promedio de $417.90
Cluster 1: Gasto promedio de $89.49
Cluster 2: Gasto promedio de $253.81



+----------+------+----------+
|id_usuario| monto|prediction|
+----------+------+----------+
|   user_58|429.28|         0|
|  user_929|217.71|         2|
|  user_783|295.41|         2|
|  user_703|101.55|         1|
|  user_786|439.36|         0|
|  user_160|157.05|         1|
|  user_155|487.57|         0|
|  user_578|236.09|         2|
|  user_574|362.67|         0|
|  user_412| 59.24|         1|
+----------+------+----------+
only showing top 10 rows


0: Clientes Premium / VIP: Los usuarios que más dinero dejan en RetailMax.

1: Clientes de Bajo Gasto: Usuarios que realizan compras pequeñas o económicas.

2: Clientes de Gasto Medio: El grupo estándar que mantiene un consumo equilibrado.
```

*Observaciones:*

1) Como se observa en los resultados finales, el modelo logró segmentar automáticamente a los usuarios (columna 'prediction'), permitiendo que el equipo de Marketing identifique grupos de gasto similares sin intervención humana.

2) Se recomienda enfocar campañas de fidelización en el Cluster 0 (clientes con gasto > $400).

Último vistazo al flujo construido:

| Etapa | Herramienta | Logro Principal |
|---|---|---|
| **Ingeniería** | RDDs | Procesamiento masivo y resiliencia (DAG). |
| **Estructura** | DataFrames | Optimización con esquemas y Spark SQL. |
| **Persistencia** | Parquet | Almacenamiento eficiente de grado industrial. |
| **Inteligencia** | MLlib | Segmentación automática de clientes (K-Means). |

## Resumen de lo logrado:

- **Limpieza de datos:** Procesaste 10,000 registros de RetailMax.
- **Big Data:** Usaste RDDs, entendiste el DAG y optimizaste con DataFrames y Parquet.
- **Análisis SQL:** Generaste métricas de ventas por categoría.
- **Machine Learning:** Segmentaste a tus clientes en 3 grupos (VIP, Promedio, Bajo Gasto) usando K-Means.